

תסריטים:

א. העלאת קובץ בשם L01.pdf לאתר הקורס.
מהמחשב אשר ה- host name (המקומי!) שלו הוא room247
אתר הקורס מנוהל על ידי מערכת ה- Moodle שמוקנת ורצה ב- server אשר ה- host name שלו
הוא online.shenkar.ac.il

ב. פניה מהמחשב room247 ל- google.gemini.com
{ host name 'גלובלי'.
ה- hist name האמיתי }

תסריט א:

מבנה ה-HTTP Request Message (Upload)

כאשר מדובר בהעלאת קובץ (Upload), ה- Chrome לא ישתמש ב- GET, אלא ב- POST. כיוון
שמדובר בקובץ PDF, הפורמט הסטנדרטי יהיה multipart/form-data.

```
POST /course/view.php?id=??? HTTP/1.1
Host: online.abc.edu
Connection: keep-alive
Content-Length: ???
Cache-Control: max-age=0
sec-ch-ua: ???
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Upgrade-Insecure-Requests: 1
Origin: https://online.abc.edu
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary???
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/???
Accept: ???
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://online.abc.edu/course/view.php?id=???
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: ???
Cookie: MoodleSession=???
```

```

-----WebKitFormBoundary???
Content-Disposition: form-data; name="repo_upload_file"; filename="L01.pdf"
Content-Type: application/pdf

%PDF-1.4 ... [Binary Data of 1.5MB] ...
-----WebKitFormBoundary???—

```

ניתוח השדות:

- **Post Path / id** :??? : לא ניתן לקבוע את ה-ID המדויק של הקורס ב-Moodle ללא גישה למסד הנתונים שלו.
- **Content-Length** :??? : הערך יהיה מעט יותר מ-1.5MB (הקובץ + ה-Boundaries של ה-Multipart). לא ניתן לחשב במדויק ללא ידיעה של ה-Overhead של הטופס.
- **/sec-ch-ua / Chrome** :??? : גרסת הדפדפן המדויקת משתנה מדי שבועיים בעדכונים אוטומטיים.
- **boundary=-----WebKitFormBoundary** :??? : זוהי מחרוזת אקראית שהדפדפן מייצר כדי להפריד בין שדות בטופס. היא ייחודית לכל Request.
- **Accept-Language** :??? : תלוי בהגדרות השפה של המרצה (en-US, he-IL וכו').
- **Cookie: MoodleSession** :??? : זהו ה-Session Token. הוא מוצפן/מגובב ונוצר מחדש בכל התחברות.

ערכי ה-MSS במערכות הפעלה שונות

ה-MSS (Maximum Segment Size) הוא כמות הנתונים המקסימלית שסגמנט TCP יכול לשאת (לא כולל Headers).

שיקולים ונימוקים	ערך MSS טיפוסי (Ethernet)	מערכת הפעלה
נגזר מה-MTU של Ethernet (1500) פחות 20 (IP Header) ו-20 (TCP Header). זהו הערך האופטימלי למניעת Fragmentation.	bytes 1460	Windows 11 Pro
זהה ל-Pro. ליבת ה-Networking (Tcpip.sys) זהה בשתי הגרסאות.	bytes 1460	Windows 11 Home
(בחיבור מקומי). לינוקס משתמשת ב-Path MTU Discovery (PMTUD) כדי לכוון זאת דינמית, אך 1460 הוא ה-Base.	bytes 1460	Linux 6.11

מערכת הפעלה	ערך MSS טיפוס (Ethernet)	שיקולים ונימוקים
macOS (Latest)	bytes 1460	מבוסס על עקרונות דומים של Berkeley Sockets. בחיבורי Wi-Fi/Fiber מודרניים זהו התקן המוסכם.

השיקולים לבחירה: הערך 1460 הוא "נקודת הקסם" שמאפשרת למלא חבילת Ethernet שלמה (1500 בתים) ללא פיצול בנתבים בדרך.
 בחירה בערך גדול יותר תגרום ל-Fragmentation (איטיות), וערך קטן יותר יגרום ל-Overhead גבוה מדי של Headers.

תיאור צעדי ה-Chrome (החלוקה המתודית)

כאן אנו צוללים למימוש:

שלב א': טרום-HTTP (תשתית והכנה)

1. **Parsing:** ה-Chrome מנתח את ה-URL ומזהה את ה-Host (online.abc.edu).
2. **DNS Resolution:** פנייה לשרת ה-DNS של האוניברסיטה. קבלת כתובת ה-IP המקומית.
3. **TCP Handshake:** הקמת חיבור (SYN, SYN-ACK, ACK) מול פורט 443 בשרת.
4. **TLS Handshake:** החלפת תעודות דיגיטליות, אימות מול ה-CA של האוניברסיטה, והסכמה על מפתחות הצפנה (Symmetric keys).

שלב ב': שליחת ה-HTTP Request (ה-Upload)

1. **Encapsulation:** ה-Chrome בונה את הודעת ה-POST (כפי שהוצג קודם).
2. **MIME Handling:** הקובץ L01.pdf נקרא מהדיסק ונארז בתוך ה-Multipart body.
3. **Segmentation:** ה-Payload (1.5MB) מחולק לסגמנטים של bytes 1460 (ה-MSS שקבענו).
4. **Encryption:** כל סגמנט מוצפן ב-TLS ונשלח מעל ה-TCP שנוצר בשלב א'.

שלב ג': המתנה וקבלת ה-Response (בצד השרת והלקוח)

1. **Server Side Processing:** ה-Moodle מקבל את הקובץ, שומר אותו ב-File system ומעדכן את ה-DB.
2. **HTTP Response:** השרת מחזיר הודעת 200 OK (או 303 See Other לצורך Redirect לדף הקורס).
3. **TCP Acknowledgement:** ה-Chrome מאשר את קבלת חבילות התגובה.

שלב ד': עיבוד לאחר קבלת התשובה

1. **DOM Update:** ה-Chrome מעבד את ה-HTML/JSON שחזר מהשרת.
2. **Rendering:** המרצה רואה על המסך הודעת "File uploaded successfully" או את הקובץ מופיע ברשימת המשאבים.

רשתות תקשורת, סמסטר ב', תשפ"ו
הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP
עמוד 3

3. **Connection Management**: כיוון שהשדה Connection: keep-alive קיים, Chrome שומר את חיבור ה-TCP פתוח למשך זמן מה (Idle) למקרה שהמרצה ירצה להעלות קובץ נוסף.

ניתוח גודל ההודעה וסגמנטציה (MB Upload1.5)

- **גודל הודעת ה-HTTP Request:**

- גוף הקובץ: 1,500,000 בתים (MB1.5).
- Headers ו-Multipart Boundaries: כ-800 עד 1,200 בתים (תלוי בכמות העוגיות וה-Agent-User).
- סך כל הנתונים להעברה (Payload): כ-1,501,000 בתים.
-

- **תהליך החישוב:**

כמות הסגמנטים נקבעת לפי חלוקת סך כל הנתונים בגודל המקסימלי שכל סגמנט יכול לשאת: 1,501,000 / 1,460 חלקי שווה בערך ל-1028.08.

- **סיכום התוצאה:**

פרוטוקול ה-TCP במחשב room247 ייצור סך הכל **1,029 סגמנטים**:

- 1,028 סגמנטים יהיו "מלאים", כלומר כל אחד מהם ישא בדיוק 1,460 בתים של נתוני אפליקציה.

- סגמנט אחד אחרון יהיה "סגמנט יתרה", והוא ישא את ה-80 בתים שנותרו (לפי החישוב המקורב לעיל).

הערה הנדסית: כל סגמנט כזה ייעטף ב-Header של TCP (בגודל 20 בתים) וב-Header של IP (בגודל 20 בתים), כך שכל חבילה שתצא לרשת ה-Ethernet/WiFi תהיה בגודל של עד 1,500 בתים (ה-MTU הסטנדרטי).

הודעת ה-HTTP Response מה-Apache

השרת יחזיר תשובה המאשרת את קבלת הקובץ.

Plaintext
HTTP/1.1 200 OK
Date: ???
Server: Apache/??? (Debian)
X-Powered-By: PHP/???
Cache-Control: ???
Content-Type: text/html; charset=utf-8
Content-Length: ???
Connection: keep-alive

<html>... Upload Successful ...</html>

ניתוח השדות:

- **Date: ???**: זמן השרת המדויק ברגע סיום העיבוד.
- **Server: Apache/??? (Debian)**: גרסת ה-Apache המדויקת תלויה בעדכוני האבטחה של מנהל השרת.
- **X-Powered-By: PHP/???/Moodle**: מבוססת PHP. הגרסה תלויה בהתקנה (למשל 8.2).
- **Content-Length: ???**: גודל דף ה-HTML שמודיע על הצלחה (לרוב כמה מאות בתים).
- **Cache-Control: Moodle: ???**: no-cache נוטה לשלוח בדפים דינמיים כדי למנוע הצגת מידע ישן.

מימוש ה-HTTP Client ב-Chrome

- **האם ממומש?** כן, באופן מלא ואוטונומי. Chrome אינו מסתמך על ה-HTTP Stack של מערכת ההפעלה (כמו WinINet ב-Windows), אלא מממש מחסנית פרוטוקולים משלו הנקראת **Network Stack** (או רכיב ה-net).
- **שפת המימוש: C++**. זהו קוד מורכב ביותר הכולל ניהול HTTP Cache, Socket Pools, ו-Prioritization.
- **גודל הקוד**: רכיב ה-net כולו מחזיק כ-1,000,000 עד 1,500,000 שורות קוד. מתוכן, המימוש הספציפי של HTTP (גרסאות 1.1, 2, ו-3) תופס כמה מאות אלפי שורות.

פרוטוקולים נוספים ב-Chrome (Client vs. Server)

Chrome הוא כמעט תמיד **Client** בלבד. הוא אינו "שרת" במובן הקלאסי, אך הוא כולל רכיבי "שרת" קטנים לצורך תקשורת פנימית (IPC) או תמיכה ב-P2P.

פרוטוקול	תפקיד (Client/Server)	שפת מימוש	גודל משוער (LOC)
DNS	Client (מימוש עצמאי, לא רק קריאות OS)	C++	כ-50,000
TLS/SSL	Client (באמצעות ספריית BoringSSL)	C / ++C	כ-300,000
QUIC / HTTP/3	Client	C++	כ-200,000
WebSockets	Handshake & Client	C++	כ-40,000
WebRTC	Peer-to-Peer (מתפקד כסטאק מלא)	C++	כ-1,000,000+
FTP	Client (הוסר ברובו בגרסאות חדשות)	C++	זניח כיום
mDNS	Client/Server (לגילוי מכשירים כמו Chromecast)	C++	כ-20,000

תסריט ב':

פניה ל- online.shenkar.ac.il

- גודל הודעת ה-HTTP Request (ל-google.gemini.com) בניגוד למקרה ה-Moodle שבו ביצענו Upload, פנייה ל-Gemini היא לרוב בקשת GET (לטעינת הממשק) או בקשת POST (שליחת שאילתה) המכילה JSON.
 - הערכה כמותית: הודעת GET טיפוסית ל-Google כוללת Headers רבים (Cookies, User-Agent, Tokens).
 - גודל משוער: 800 עד 2,500 בתים.

רשתות תקשורת, סמסטר ב', תשפ"ו
הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP
עמוד 6

- **הערה:** אם מדובר בבקשת POST עם שאילתה ארוכה, הגודל יגדל בהתאם לנפח הטקסט ב-JSON, אך עבור "הודעת בקשה סטנדרטית", אלו סדרי הגודל.

2. כמות סגמנטים (Windows 11 Pro)

כפי שביססנו, ה-MSS ב-Windows 11 הוא 1460 bytes.

- אם גודל ההודעה הוא 2,000 בתים: TCP יצור 2 סגמנטים.
- הראשון יהיה בגודל מלא (1460 בתים של נתוני אפליקציה) והשני יכיל את היתרה (540 בתים).

3. חובת הקמת קישור (Connection Establishment)

כן, חד משמעית. פרוטוקול TCP הוא Connection-oriented.

לפני שבית אחד של HTTP יוצא מה-Chrome, ה-Kernel של Windows חייב להשלים את תהליך ה-Three-Way Handshake מול השרת. ללא קישור פעיל במצב ESTABLISHED, ניסיון לשלוח נתונים יידחה ברמת ה-Stack.

4. ממשק ה-WinSock API: מול BSD Sockets

במחשב room247 המריץ Windows, הקריאות מתבצעות ל-WinSock (Windows Sockets 2).

- **הנימוק:** למרות ש-WinSock מבוסס היסטורית על מודל ה-BSD Sockets (כדי להקל על פורטינג של קוד), הוא מימוש קנייני של מיקרוסופט (הספריה ws2_32.dll).
- **הבדלים קריטיים:** WinSock כולל הרחבות ספציפיות לביצועים ב-Windows (כמו AcceptEx או ConnectEx) התומכות במודל ה-I/O Completion Ports (IOCP). Chrome משתמש בהרחבות אלו כדי לנהל אלפי חיבורים בצורה אסינכרונית ויעילה הרבה יותר ממה שממשק ה-BSD הסטנדרטי מאפשר.

5. צעדי ה-TCP ברמת ה-Sockets (הקמת קישור)

התהליך מתבצע באמצעות סדרת קריאות מערכת (System Calls) מצד ה-Chrome ל-WinSock:

1. **socket():** מבקש מה-Kernel להקצות משאב (Handle) עבור סוקט חדש מסוג AF_INET (IPv4/v6) ו-SOCK_STREAM (TCP).
2. **bind():** (אופציונלי): לרוב Chrome לא קורא לזה ב-Client; ה-Kernel בוחר פורט מקור אקראי (Ephemeral Port).
3. **connect():** זוהי קריאת המפתח. Chrome מספק את ה-IP של גוגל ואת פורט 443.
 - ה-Kernel שולח חבילת SYN.
 - השרת מחזיר SYN-ACK.
 - ה-Kernel שולח ACK.
 - הקריאה connect() חוזרת ל-Chrome עם תשובת "Success".
4. **setsockopt():** Chrome קורא לזה כדי להגדיר פרמטרים כמו TCP_NODELAY (ביטול אלגוריתם Nagle) כדי להבטיח שהודעת ה-HTTP תצא מיד ללא השהיה.

6. העברת הודעת ה-HTTP מה-Client ל-TCP

לאחר שהקישור הוקם, ה-HTTP Client ב-Chrome (שנמצא ב-User Space) "מחזיק" את הודעת ה-Request בזיכרון (Buffer).

- **הקריאה:** Chrome קורא לפונקציה **send()** (או WSASend ב-WinSock האסינכרוני).
- **התהליך:**

1. ה-Chrome מעביר ל-Kernel מצביע (Pointer) ל-Buffer של הודעת ה-HTTP ואת גודלה.
2. מתבצע **Context Switch** מה-User Mode ל-Kernel Mode.
3. ה-Kernel מעתיק את הנתונים מהזיכרון של Chrome אל ה-TCP Send Buffer בתוך ה-Kernel.
4. מרגע זה, ה-TCP Stack של Windows לוקח פיקוד: הוא מבצע את ה-Segmentation (לפי ה-MSS), מוסיף Headers, ומוריד את החבילות לשכבת ה-IP.

הערה:

עלינו להבחין בין ה-MTU (Maximum Transmission Unit) של השכבה השנייה לבין ה-MSS של TCP. הערך 1500 הוא אכן ה-MTU הסטנדרטי של Ethernet, והוא הפך ל"מכנה המשותף הנמוך ביותר" של האינטרנט.

השפעת ה-Options ב-TCP וב-IP על ה-MSS

כאן טמון ההבדל בין MSS סטטי לבין MSS דינמי:

- **TCP Options:** אם TCP משתמש ב-Options (כמו Timestamp או Selective ACK), גודל ה-Header אכן גדל מעבר ל-20 בתים. במקרה כזה, ה-Stack של מערכת ההפעלה (Windows או Linux) יקטין את כמות הנתונים שהוא מכניס לכל סגמנט כדי שסך הכל (Data + Headers) לא יחרוג מה-MTU.
 - לדוגמה: אם ה-TCP Header הוא 32 בתים, ה-Payload יהיה 1448 בתים בלבד, כדי שהחבילה הסופית תישאר 1500.
 - **IP Options:** כמעט ולא נעשה בהם שימוש באינטרנט המודרני (מסיבות אבטחה וביצועים בנתבים). אם הם קיימים, התוצאה זהה: ה-MSS האפקטיבי קטן.
- התובנה:** ה-MSS הוא ה"יתרה" שנשארת אחרי שמורידים את כל ה-Headers מה-MTU.

MSS סטטי (The Advertised MSS)

- הגדרה:** ה-MSS הסטטי הוא הערך המקסימלי התיאורטי שצד אחד בשיחה מוכן לקבל. ערך זה נקבע פעם אחת בלבד – במהלך ה-Three-Way Handshake של TCP.
- **המכניקה:** כאשר מחשב ה-room247 שולח חבילת SYN, הוא כולל בתוך ה-TCP Options שדה שנקרא "MSS Option". הוא מצהיר: "ה-MTU שלי הוא 1500, לכן ה-MSS הסטטי שלי הוא 1460".
 - **מקור הערך:** הוא נגזר ישירות מהגדרות כרטיס הרשת (NIC) במערכת ההפעלה. Windows 11 מסתכלת על ה-MTU המוגדר לממשק (Interface) ומחסירה ממנו 40 בתים (Header קבוע של IP ו-TCP).
 - **המשמעות:** זהו "גבול עליון" קשיח. השרת של גוגל לעולם לא ישלח סגמנט הגדול מ-1460 בתים למחשב שלך, כי הוא יודע שזה יגרום לפיצול (Fragmentation) אצלך.

MSS דינמי (The Effective MSS)

הגדרה: ה-MSS הדינמי (או ה-Effective MSS) הוא הגודל המשתנה של ה-Payload בכל סגמנט וסגמנט במהלך השיחה. הוא כמעט תמיד יהיה קטן או שווה ל-MSS הסטטי.

- **למה הוא "דינמי"? כי הוא מושפע משימוש באופציות נוספות בכל חבילה וחבילה.**
 - **TCP Options:** אם בסגמנט מסוים TCP מחליט להשתמש ב-Timestamps (עוד 12 בתים), ה-MSS הדינמי לאותו סגמנט יקטן ל-1448.
 - **IP Options / Extensions:** ב-IPv6, אם יש Extension Headers, ה-MSS הדינמי יקטן משמעותית כדי "לפנות מקום" לכותרות בתוך מסגרת ה-MTU.
 - **המימוש ב-Kernel:** ה-TCP Stack מחשב בכל פעם שפונקציית ה-send() נקראת:
$$\text{Effective MSS} = \text{MTU} - \text{Actual IP Headers} - \text{Actual TCP Headers}$$
-

נניח כעת לצורך המשך הדיון:

פניה מ-room247 ל-google.gemini.com

הודעת GET. לא POST.

גודל הודעה: 2500 בתים

TCP

1. האם ה-HTTP Client יעביר אותה בשלמותה בקריאה אחת?

כן, סביר מאוד. ה-HTTP Client (בתוך ה-Network Service של Chrome) מנהל Buffer בזיכרון של ה-User Space. מכיוון ש-2500 בתים הם נפח זניח עבור ה-RAM של מחשב מודרני, Chrome יבנה את כל הודעת ה-HTTP (Headers + Cookies) ב-Buffer אחד ויבצע קריאה אחת ל-WSASend (או send).

חשוב להבין: הקריאה ל-WSASend היא במישור ה-User-to-Kernel. ה-Chrome אומר למערכת ההפעלה: "הנה 2500 בתים, תעבירי אותם לצד השני". החלוקה לסגמנטים (Segmentation) אינה מתבצעת ב-Chrome, אלא ב-TCP Stack שבתוך ה-Windows Kernel.

2. פירוט שדות ה-TCP Header (גודל בסיסי: 20 בתים)

הכותרת של TCP אחראית על אמינות, סדר וניהול זרימה. להלן השדות:

- **Source Port (16 bit):** פורט המקור במחשב room247 (פורט אקראי/Ephemeral מעל 1024). מאפשר ל-OS לדעת לאיזו אפליקציה (Chrome) להחזיר את התשובה.

רשתות תקשורת, סמסטר ב', תשפ"ו

הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP

עמוד 9

- **Destination Port (16 bit)**: פורט היעד. במקרה של HTTPS ל-Gemini, הערך יהיה 443.
- **Sequence Number (32 bit)**: מספר הרצף של הבית הראשון בסגמנט זה. מאפשר לצד השני לסדר את החבילות בסדר הנכון.
- **Acknowledgment Number (32 bit)**: רלוונטי אם דגל ה-ACK דולק. מציין את מספר הבית הבא שהשולח מצפה לקבל מהצד השני.
- **Data Offset (4 bit)**: מציין את גודל ה-Header במילים של 32 סיביות. עוזר לדעת איפה מתחילים הנתונים (ה-Payload).
- **Reserved (3 bits)**: שמור לשימוש עתידי (תמיד אפס).
- **Flags (9 bits)**: דגלי בקרה (SYN, ACK, FIN, RST, PSH, URG). למשל, בשידור נתונים דגל ה-PSH (Push) לרוב ידלק כדי להורות לשרת להעביר את הנתונים לאפליקציה מיד.
- **Window Size (16 bit)**: ניהול זרימה (Flow Control). מציין כמה בתים השולח מוכן לקבל מהצד השני לפני שיעצור.
- **Checksum (16 bit)**: בדיקת תקינות. מאפשר לשרת לדעת אם חלו שיבושים בביטים במהלך המעבר ברשת.
- **Urgent Pointer (16 bit)**: בשימוש נדיר מאוד. מציין אם יש נתונים דחופים.
- **Options (0-40 bytes)**: שדות אופציונליים כמו MSS, Timestamps, או SACK.

3. תוכן 2 הסגמנטים שיישלחו מ-room247

בהינתן הודעה של 2500 בתים ו-MSS של 1460, ה-Kernel יפצל את ההודעה לשני סגמנטים:

סגמנט 1: הסגמנט המלא

- **TCP Header (20 bytes)**
 - **Source Port**: ??? (נקבע דינמית על ידי Windows בעת פתיחת הסוקט).
 - **Dest Port**: 443.
 - **Sequence Number**: X (המספר ההתחלתי שנקבע ב-Handshake).
 - **Flags**: ACK, PSH.
- **Payload (1460 bytes)**: מכיל את תחילת הודעת ה-HTTP GET (שורת הבקשה ורוב ה-Headers).

סגמנט 2: סגמנט היתרה

- **TCP Header (20 bytes)**
 - **Source Port**: זהה לסגמנט 1.
 - **Dest Port**: 443.
 - **Sequence Number**: X + 1460 (הבית הבא ברצף).
 - **Flags**: ACK, PSH.
 - **Payload (1040 bytes)**: מכיל את יתרת הודעת ה-HTTP (סיומת ה-Headers וה-Cookies). החישוב: $1040 = 2500 - 1460$.
- למה לא ניתן לקבוע חלק מהערכים?
1. **Source Port**: נקבע על ידי ה-Ephemeral Port range של Windows (לרוב 49152 עד 65535). ללא ריצה חיה, לא ניתן לדעת איזה פורט פנוי המערכת בחרה.
 2. **Sequence Number**: לפי התקן (RFC 6528), מספר הרצף ההתחלתי (ISN) חייב להיות אקראי כדי למנוע התקפות ניחוש (TCP Sequence Prediction).

3. **Checksum**: מחושב על בסיס הנתונים הפיזיים. שינוי של ביט אחד ב-IP או בנתונים ישנה את הערך.
4. **Acknowledgment Number**: תלוי בנתונים שגוגל כבר שלחה בחזרה (למשל במהלך ה-TLS Handshake), נתון שמשתנה בכל שיחה.

IPv4

כשאנחנו יורדים משכבת התחבורה (Transport) לשכבת הרשת (Network), אנחנו עוברים מניהול ה"שיחה" (Flow Control) לניהול ה"חבילה". במחשב room247, התהליך הזה מתבצע כולו בתוך ה-Kernel של Windows 11.

1. איך TCP "מעביר" את הסגמנטים ל-IP?

- בהנדסת תוכנה של מערכות הפעלה, המעבר אינו מתבצע על ידי "שליחה" פיזית של עותק נתונים, אלא על ידי העברת מצביעים (Pointers) בזיכרון ה-Kernel.
- **התהליך**: ברגע ש-TCP מסיים לבנות את הסגמנט (כולל ה-TCP Header), הוא קורא לפונקציה פנימית ב-Network Stack (למשל IpSendDatagram או פונקציה מקבילה ב-Windows Networking Stack).
 - **העברת מבנה נתונים**: TCP מעביר ל-IP מבנה נתונים (ב-Windows זהו לרוב מבנה מסוג NET_BUFFER) הכולל מצביע למיקום הסגמנט בזיכרון ואת כתובת היעד.
 - **Encapsulation**: שכבת ה-IP לא משנה את נתוני ה-TCP. היא פשוט "מדביקה" (Prefix) את ה-IP Header לפני ה-TCP Header בזיכרון, ובכך יוצרת את ה-Datagram.

2. תוכן 2 ה-IPv4 Datagrams (במחשב room247)

כל Datagram מכיל Header של 20 בתים (ללא אופציות) ואת הסגמנט של TCP.

Datagram 1 (נושא את סגמנט 1):

- **Version**: 4.
- **IHL (Header Length)**: 5 (מייצג 20 בתים).
- **DSCP/ECN**: 0 (לרוב ברירת מחדל לתעבורת Web רגילה).
- **Total Length**: 1480 (1460 של TCP Payload + 20 של TCP Header + 20 של Header).
- **Identification**: ??? (מספר ייחודי שנקבע על ידי ה-OS לזיהוי החבילה).
- **Flags / Fragment Offset**: 0 (ביט ה-Don't Fragment לרוב דולק ב-Windows כדי למנוע Fragmentation בדרך).
- **TTL (Time to Live)**: 128 (ערך ברירת המחדל הטיפוסי של Windows 11).
- **Protocol**: 6 (מזהה את הפרוטוקול העליון כ-TCP).
- **Header Checksum**: ??? (מחושב דינמית על ה-Header בלבד).
- **Source IP**: ??? (כתובת ה-IP המקומית של room247).
- **Destination IP**: ??? (כתובת ה-IP הציבורית של google.gemini.com).

Datagram 2 (נושא את סגמנט 2):

- **Total Length** : 1060 (1040 של 20 + TCP Payload של 20 + TCP Header של IP (Header).
- **Identification** : ??? (עוקב ל-Datagram 1).
- **יתר השדות (TTL, Protocol, IPs)** : זהים ל-Datagram 1.

3. תוכן 2 ה-IPv6 Datagrams (במחשב room247)

ב-IPv6 המבנה קבוע ופשוט יותר, אך ה-Header גדול פי 2 (40 בתים).

Datagram 1 (IPv6)

- **Version** : 6.
- **Traffic Class** : 0.
- **Flow Label** : ??? (מספר אקראי בן 20 ביט המשמש לזיהוי הזרם בנתבים).
- **Payload Length** : 1480 (ב-IPv6 אורך ה-Payload כולל את ה-TCP Header והנתונים בלבד).
- **Next Header** : 6 (TCP).
- **Hop Limit** : 128 (מקביל ל-TTL).
- **Source IPv6 Address** : ??? (כתובת ה-IPv6 של room247).
- **Destination IPv6 Address** : ??? (כתובת ה-IPv6 של google.gemini.com).

Datagram 2 (IPv6)

- **Payload Length** : 1060.
- **יתר השדות** : זהים ל-Datagram 1 (למעט ה-Flow Label שעשוי להיות זהה לצורך ניתוב עקבי).

4. ההבדלים העיקריים והסיבות להם

מאפיין	IPv4	IPv6	הסבר טכני
גודל Header	20 בתים	40 בתים	IPv6 ויתר על שדות משתנים לטובת עיבוד מהיר יותר בנתבים, אך כתובות ה-IP ארוכות בהרבה (128 ביט).
Checksum	קיים ב-Header	הוסר	ב-IPv6 סומכים על שכבות 2 (Ethernet) ו-4 (TCP) שיבצעו בדיקת תקינות. זה חוסך זמן עיבוד יקר לכל נתב בדרך.
Fragmentation	מותר לנתבים לפצל	אסור לנתבים	ב-IPv6 רק המקור (room247) רשאי לפצל. אם חבילה גדולה מדי, הנתב זורק אותה ושולח הודעת ICMPv6.

רשתות תקשורת, סמסטר ב', תשפ"ו

הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP

עמוד 12

מאפיין	IPv4	IPv6	הסבר טכני
Total Length שדה	כולל את ה-Header	Payload Only	ב-IPv6 שדה האורך מתייחס רק למה שבא אחרי ה-Header הבסיסי, מה שמפשט את החישוב.

למה לא ניתן לקבוע את הערכים?

1. כתובות IP: תלויות ב-ISP ובניתוב ה-DNS ברגע הנתון.
2. Identification (v4) / Flow Label (v6): אלו ערכים אקראיים שנוצרים ב-Kernel למניעת התקפות ניחוש ולייעול ניתוב.
3. Checksum (v4): משתנה בכל פעם ששדה TTL יורד בנתב בדרך, לכן הוא תלוי-זמן ומרחק.
4. Payload Length (v6): למרות שחישבנו אותו, אם יתווספו Extension Headers (אופציונלי), הערך ישתנה.

IPv6

זוהי שאלת "המפתח" שסטודנטים רבים מתבלבלים בה, והיא חושפת את הדינמיות של פרוטוקול ה-TCP. התשובה הקצרה היא: ה-MSS ב-IPv6 פשוט אינו 1460 בתים. כדי לשמור על ה-MTU של ה-Ethernet בגודל 1500, מערכת ההפעלה חייבת לבצע התאמה של ה-MSS בהתאם לגרסת ה-IP שבה היא משתמשת.

להלן הניתוח ההנדסי של ה"חשבונאות" בתוך ה-Kernel:

1. החישוב ב-IPv4 (המודל הסטנדרטי)

כפי שראינו, ב-IPv4 הכותרות קטנות יותר:

- MTU: 1500 bytes
- IP Header: 20 bytes
- TCP Header: 20 bytes
- היתרה לנתונים (MSS): $1500 - 20 - 20 = 1460$ bytes.

2. החישוב ב-IPv6 (המודל המורחב)

בגלל שכתובות ה-IP בגרסה 6 הן בנפח 128 סיביות כל אחת, הכותרת הבסיסית (Fixed Header) צמחה ל-40 בתים. כדי לא לחרוג מה-MTU של ה-Ethernet (שנשאר 1500), ה-Kernel מקטין את ה-MSS:

- MTU: 1500 bytes
- IPv6 Header: 40 bytes
- TCP Header: 20 bytes
- היתרה לנתונים (MSS): $1500 - 40 - 20 = 1440$ bytes.

3. איך זה "מסתדר" ב-room247?

רשתות תקשורת, סמסטר ב', תשפ"ו
הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP
עמוד 13

בזמן הקמת הקשר (TCP Handshake), ה-Kernel ב-Windows 11 בודק באיזה פרוטוקול הוא משתמש:

1. **בשלב ה-SYN:** אם ה-Chrome מנסה להתחבר לכתובת IPv6 של Google, ה-Kernel ישלח בתוך ה-TCP Options ערך $MSS = 1440$.
 2. **הסכמה הדדית:** השרת של גוגל יראה את הערך הזה וידע שאסור לו לשלוח סגמנטים עם יותר מ-1440 בתים של נתוני אפליקציה, אחרת החבילה תהיה גדולה מ-1500 ותיזרק בנתבי הדרך (כי כפי שצינו, ב-IPv6 אסור לנתבים לבצע Fragmentation).
 4. **המשמעות עבור הודעת ה-GET (2500 בתים)**
אם נחזור לתסריט שלך תחת IPv6, חלוקת הסגמנטים תשתנה מעט:
 - **סגמנט 1:** ישא 1440 בתים של נתוני HTTP.
 - **סגמנט 2:** ישא את היתרה — 1060 בתים (2500 – 1440).
 - **סה"כ:** עדיין 2 סגמנטים, אך ה"אריזה" הפנימית שלהם שונה כדי לפנות מקום לכותרת ה-IP הגדולה יותר.
- לסיכום:** ה-MSS אינו מספר קבוע ב-Windows, אלא תוצאה של חיסור הכותרות מה-MTU. ב-Windows 11 Pro, ה-Stack "חכם" מספיק כדי לזהות את סוג ה-IP ולהכריז על ה-MSS הנכון כבר בבית הראשון של השיחה.

להלן ההסבר לגבי ההתאמה בין ה-MSS לבין גרסאות ה-IP השונות, בהתבסס על MTU סטנדרטי של 1500 בתים ב-Ethernet:

1. החישוב ב-IPv4

בפרוטוקול IPv4, הכותרות תופסות פחות מקום בתוך החבילה:

- גודל חבילה מקסימלי 1500 (MTU): בתים.
- כותרת IP (ללא אופציות): 20 בתים.
- כותרת TCP (ללא אופציות): 20 בתים.
- היתרה שנותרה לנתוני האפליקציה 1500 (MSS): פחות 20 פחות 20, שווה ל-1460 בתים.

2. החישוב ב-IPv6

בפרוטוקול IPv6, הכותרת הבסיסית גדולה יותר באופן משמעותי כדי להכיל כתובות באורך 128 סיביות:

- גודל חבילה מקסימלי 1500 (MTU): בתים (נשאר ללא שינוי כדי להתאים ל-Ethernet).
- כותרת IPv6 (קבועה): 40 בתים.
- כותרת TCP (ללא אופציות): 20 בתים.
- היתרה שנותרה לנתוני האפליקציה 1500 (MSS): פחות 40 פחות 20, שווה ל-1440 בתים.

3. איך זה "מסתדר" במחשב room247?

ה-MSS אינו ערך קשיח שנקבע מראש לכל המצבים, אלא ערך שמערכת ההפעלה (Windows 11 Pro) מחשבת באופן דינמי בהתאם לסוג החיבור:

- כאשר ה-Chrome פותח חיבור לכתובת IPv6, ה-TCP Stack ב-Kernel מזהה שהכותרת תהיה בת 40 בתים.
- לכן, כבר בשלב ה-Handshake (בחבילת ה-SYN), המחשב יצהיר על MSS של 1440 בתים.
- כך מובטח שהחבילה הסופית (נתונים + כותרת TCP + כותרת IPv6) תהיה בדיוק 1500 בתים ולא תחרוג מהמגבלה של ה-Ethernet.

4. המשמעות עבור הודעת ה-GET (2500 בתים) תחת IPv6:

במקרה זה, החלוקה לשני סגמנטים תתבצע כך:

- סגמנט 1: ישא 1440 בתים של נתוני HTTP.
- סגמנט 2: ישא את היתרה — 1060 בתים (2500 פחות 1440).
- כל אחד מהסגמנטים הללו ייארז בתוך כותרת IPv6 של 40 בתים, מה שיביא את החבילה הראשונה לגודל מקסימלי של 1500 בתים בדיוק.

סיכום: מערכת ההפעלה מבצעת "חשבונאות" של בתים. היא מקטינה את ה-MSS ככל שכותרת ה-IP גדלה, כדי להבטיח שהתוצאה הסופית תמיד תתאים ל"צינור" הפיזי של 1500 בתים.

איך בדיוק מערכת ההפעלה מבצעת התאמה של ה-MSS בהתאם לגרסת ה-IP:

התהליך שבו מערכת ההפעלה (במקרה שלנו Windows 11 Pro במחשב room247) מבצעת את ההתאמה בין ה-MSS לבין ה-IP הוא מלאכת מחשבת של ניהול שכבות בתוך ה-Kernel. הפעולה אינה מתבצעת ברמת האפליקציה (Chrome), אלא עמוק בתוך ה-TCP/IP Stack (הקובץ tcpip.sys).

להלן פירוט שלבי התהליך "מתחת למכסה המנוע":

1. שלב ה-Interface Discovery (גילוי הממשק)

ברגע שנוצר Socket חדש, ה-Kernel משייך אותו לממשק רשת פיזי (NIC - Network Interface Card). לכל ממשק כזה ב-Windows יש ערך שנקרא MTU (Maximum Transmission Unit) המוגדר ב-Registry או נקבע דינמית.

- עבור Ethernet ו-WiFi, הערך הוא כמעט תמיד 1500 בתים.
- ה-Kernel שומר את הערך הזה במבנה נתונים פנימי של הממשק.

2. קביעת ה-IP Version (ההחלטה על ה-Protocol)

כאשר Chrome קורא ל-connect(), הוא מספק כתובת יעד. ה-Stack בודק האם מדובר בכתובת IPv4 או IPv6. זהו רגע המפתח:

- אם IPv4: ה-Kernel יודע שה-Header הבסיסי הוא 20 בתים.
- אם IPv6: ה-Kernel יודע שה-Header הבסיסי הוא 40 בתים.

3. חישוב ה-Calculated MSS (בזמן ה-Handshake)

לפני שליחת חבילת ה-SYN הראשונה, ה-TCP Stack מבצע את החישוב הבא :

1. הוא לוקח את ה-MTU של הממשק (1500).
2. הוא מחסיר את גודל ה-Fixed IP Header (20 או 40).
3. הוא מחסיר את גודל ה-Fixed TCP Header (20 בתים).
4. **התוצאה:** 1460 (עבור IPv4) או 1440 (עבור IPv6).

ערך זה מוכנס לשדה ה-MSS Option בתוך כותרת ה-TCP של חבילת ה-SYN. זהו ה-Advertised MSS.

4. שלב ה-Sizing האופרטיבי (הטיפול ב-Options)

כאן העסק הופך למורכב יותר. בכל פעם שה-TCP Stack מקבל נתונים מהאפליקציה (ה-2500 בתים של הודעת ה-GET), הוא צריך לבנות סגמנט פיזי. הוא לא מסתמך רק על החישוב הסטטי, אלא מבצע בדיקה בכל חבילה :

- **בדיקת Options:** ה-Stack בודק אילו אופציות TCP פעילות לסשן הזה. לדוגמה, אם מופעל TCP Timestamps (נפוץ מאוד ב-Windows 11), מתווספים **12 בתים** נוספים לכל Header של TCP.
- **חישוב היתרה:** ה-Kernel מחשב את ה-Effective MSS (eMSS):
$$eMSS = MTU - IP\ Header\ Size - TCP\ Header\ Size\ With\ Options$$

5. הזרמת הנתונים (Fragmentation avoidance)

כעת, כשיש ל-Kernel את ה-eMSS המדויק (נניח 1448 בתים ב-IPv4 עם Timestamps) :

1. ה-Stack לוקח את ה-Buffer שקיבל מ-Chrome.
2. הוא "גוזר" ממנו נתונים בדיוק בגודל ה-eMSS.
3. הוא בונה את ה-Header עם ה-Options, מצמיד את הנתונים, ושולח לשכבת ה-IP.
4. **התוצאה הסופית:** החבילה כולה (L3) שוקלת בדיוק 1500 בתים.

6. מנגנון ה-PMUD (התאמה לסביבה החיצונית)

גם אם ה-Kernel ב-room247 החליט על 1460, ייתכן שבדרך ל-Gemini יש נתב (Router) עם MTU קטן יותר (למשל 1492 בגלל מנהרת PPPoE).

- אם חבילה של 1500 מגיעה לנתב כזה, הנתב זורק אותה ושולח הודעת **ICMP "Packet Too Big"**.
- ה-Kernel ב-Windows מקבל את ההודעה, מעדכן את מבנה הנתונים של ה-Routing Table עבור היעד הספציפי הזה, ומקטין את ה-MSS לשיחה הזו בלבד באופן דינמי. תהליך זה נקרא **Path MTU Discovery**.
- **לסיכום:** מערכת ההפעלה מבצעת התאמה זו על ידי שילוב של ערכים קבועים (MTU של הכרטיס), לוגיקה של הפרוטוקול (v4/v6), ומשוב חי מהרשת (ICMP). הכל קורה בשכבת ה-Kernel, כך שה-Chrome "רואה" צינור נתונים רציף ואינו מודע כלל לחשבונאות הבתים המתרחשת מתחתיו.

שכבת הערוץ [Data Link Layer]

{ממומשת בחומרה !}

{NIC -ב}

כשאנחנו יורדים לשכבה השנייה (Data Link Layer), אנחנו יוצאים מהעולם הלוגי של כתובות ה-IP ועוברים לעולם הפיזי של כתובות ה-MAC והתקשורת על גבי הכבל (או האוויר). במחשב room247, שכבת ה-IP מעבירה את ה-Datagram (החבילה) אל מנהל ההתקן (Driver) של כרטיס הרשת (NIC). להלן הניתוח המפורט של אריזת ה-Datagrams בתוך **Ethernet II Frame**:

1. המעבר מ-IP ל-Ethernet: תהליך ה-Encapsulation

ברגע ששכבת ה-IP מסיימת לבנות את ה-Datagram, היא צריכה לדעת "למי" לשלוח אותו ברמה הפיזית.

- **ARP / NDP**: אם מדובר ב-IPv4, המחשב משתמש בפרוטוקול **ARP** כדי למצוא את כתובת ה-MAC של היעד (או של ה-Default Gateway). ב-IPv6, התהליך מתבצע באמצעות **NDP** (Neighbor Discovery Protocol).
- **העברה ל-Driver**: לאחר שנמצאה כתובת ה-MAC, ה-Kernel מצמיד את ה-Ethernet Header ל-Datagram ומעביר אותו לכרטיס הרשת.

2. מבנה ה-Ethernet II Frame (שכבה 2)

המסגרת (Frame) עוטפת את ה-Datagram משני צדדיו:

1. **SFD (8 bytes) & Preamble**: אלו בתים שנועדו לסנכרון פיזי של השעונים בין הכרטיסים. הם לא נחשבים חלק מה-MTU.
2. **Destination MAC Address (6 bytes)**: כתובת ה-MAC של התחנה הבאה (בדרך כלל ה-Switch של האוניברסיטה או הנתב).
3. **Source MAC Address (6 bytes)**: כתובת ה-MAC הייחודית של כרטיס הרשת במחשב room247.
4. **EtherType (2 bytes)**: שדה קריטי המציין איזה פרוטוקול נמצא בתוך ה-Frame:
 - עבור **IPv4**, הערך הוא 0x0800.
 - עבור **IPv6**, הערך הוא 0x86DD.
5. **Payload (46-1500 bytes)**: כאן נמצא ה-IP Datagram בשלמותו (כולל ה-TCP וה-HTTP).
6. **FCS - Frame Check Sequence (4 bytes)**: קוד CRC לבדיקת שגיאות. אם החבילה השתבשה בדרך, ה-FCS לא יתאים והכרטיס המקבל יזרוק את החבילה.

3. תוכן 2 ה-Ethernet Frames שיישלחו (Plain Text)

נניח שאנחנו עובדים עם **IPv4** עבור הודעת ה-GET (2500 בתים):

Frame 1 (נושא את ה-Datagram הראשון):

- **Dest MAC**: ??? (כתובת ה-MAC של הנתב הארגוני - ה-Default Gateway).
- **Source MAC**: ??? (כתובת ה-MAC של room247).
- **EtherType**: 0x0800 (IPv4).
- **Payload**: ה-IP Datagram הראשון (1480 בתים).
 - פירוט ה-Payload: (20B HTTP Data + 1440B TCP Header + 20B IP Header).

רשתות תקשורת, סמסטר ב', תשפ"ו

הסבר האינטראקציה בין השכבות השונות במודל ה-TCP/IP

עמוד 17

- הערה: השתמשתי ב-1440 לנתונים כי הנחנו קודם שימוש ב-TCP Options כמו Timestamps המוריד את ה-MSS ל-1440.
- FCS: ??? (מחושב על ידי החומרה של כרטיס הרשת בזמן השידור).
- סך הכל גודל ה-Payload בשכבה 2: 1480 בתים (פחות מה-MTU של 1500).
- Frame 2 (נושא את ה-Datagram השני):
 - Dest MAC: זהה ל-Frame 1.
 - Source MAC: זהה ל-Frame 1.
 - EtherType: 0x0800 (IPv4).
 - Payload: ה-IP Datagram השני (1060 בתים).
- פירוט ה-Payload: (20B HTTP Data + 20B TCP Header + B IP Header).
- FCS: ???
- סך הכל גודל ה-Payload בשכבה 2: 1060 בתים.

4. למה לא ניתן לקבוע את הערכים?

1. כתובות MAC: כתובת ה-Source צרובה בחומרה של המחשב ב-room247. כתובת ה-Dest תלויה בצידוד הרשת הספציפי באוניברסיטה (Cisco, Juniper וכו').
2. FCS: זהו חישוב מתמטי (Cyclic Redundancy Check) על כל הביטים ב-Frame. כל שינוי קטן ב-IP Identification או ב-TCP Sequence Number ישנה את ה-FCS לחלוטין.
3. Preamble: לרוב לא נראה אותו בתוכנות כמו Wireshark כי הוא מוסר על ידי כרטיס הרשת לפני שהמידע מועבר ל-OS.

5. הבדלים במקרה של IPv6

אם ה-Datagram הוא מסוג IPv6, ההבדל היחיד ב-Frame יהיה:

- שדה ה-EtherType ישתנה ל-0xDD0x86.
- ה-Payload יהיה ה-IPv6 Datagram (עם ה-Header של ה-40 בתים).
- גודל ה-Payload של Frame 1 עדיין יהיה עד 1500 בתים (למשל: 40 + 20B TCP + B IP + 1440B Data).

סיכום הזרימה ב-room247:

ה-Chrome שלח 2500 בתים. ה-TCP פיצל אותם לשניים. ה-IP הוסיף כתובות. ה-Ethernet הוסיף כתובות MAC ובדיקת שגיאות פיזית. עכשיו, שני ה-Frames האלו יוצאים פיזית כפולסים של אור בסיב האופטי או כאותות חשמליים בכבל ה-Copper אל עבר ה-Switch הקרוב. האם נרצה כעת לנתח את הצעד הבא – מה קורה בתוך ה-Switch האוניברסיטאי כשהוא מקבל את ה-Frame הזה? איך הוא יודע לאן להעביר אותו על סמך טבלת ה-MAC שלו?

האינטראקציה:

- בין ה-IP, (בין אם v4 או v6)

- לבין ה-device driver של כרטיס הרשת

- לבין כרטיס הרשת

כדי להבין את האינטראקציה הזו ברמה העמוקה ביותר, עלינו לנתח את ה"קרביים" של מערכת ההפעלה Windows (ה-Kernel) ואת האופן שבו חומרה ותוכנה מסנכרות נתונים ללא עירוב מתמיד של המעבד המרכזי.
להלן פירוט של שלוש נקודות הממשק הקריטיות:

- 1. הממשק הלוגי: מה-IP אל ה-NDIS (The Hand-off)**
בתוך ה-Kernel של Windows 11, קיימת הפרדה מוחלטת בין פרוטוקול הרשת (IP) לבין החומרה. המתווך הוא ה-NDIS (Network Driver Interface Specification).
 - **מבנה ה-NBL (Net Buffer List):** ה-IP לא מעביר "קובץ". הוא יוצר מבנה נתונים מורכב שנקרא **NBL**. דמיין זאת כמעטפה שבה נמצאים:
 - **MDL (Memory Descriptor List):** רשימה של כתובות זיכרון פיזיות שבהן נמצאים שברי החבילה. החבילה לא חייבת להיות רציפה בזיכרון ה-RAM; ה-Header יכול להיות בכתובת אחת והנתונים (Payload) בכתובת אחרת.
 - **OOB Data (Out-Of-Band):** מידע שאינו נשלח ברשת אלא משמש את הדרייבר (למשל: "חשב Checksum עבור חבילה זו בחומרה").
 - **הקריאה לפונקציה:** ה-IP קורא ל-`NdisSendNetBufferLists`. ברגע זה, הבעלות על הזיכרון עוברת מה-Stack אל ה-NDIS Core.

- 2. הממשק התוכנתי: מה-NDIS אל ה-Miniport Driver**
ה-Miniport Driver הוא ה"מתרגם" הספציפי של יצרן הכרטיס (למשל Intel). כאן מתרחש ניהול התורים.
 - **ניהול ה-TX Ring (Transmit Ring):** הדרייבר מקצה אזור בזיכרון ה-RAM שמשותף לו ולכרטיס הרשת. אזור זה מאורגן כ"טבעת" של תאים (Descriptors).
 - **מילוי ה-Descriptors:** הדרייבר לוקח את ה-NBL שקיבל מה-NDIS וכותב לתוך הטבעת את הכתובת הפיזית של הנתונים. הוא לא מעתיק את הנתונים! הוא רק כותב "הנתונים נמצאים בכתובת 123450X".
 - **Memory Barrier:** הדרייבר חייב להשתמש בפקודות מעבד מיוחדות כדי להבטיח שהנתונים נכתבו ל-RAM לפני שהוא מודיע לכרטיס הרשת שהם שם. זהו שלב קריטי במערכות מרובות ליבות (Symmetric Multiprocessing).

- 3. הממשק הפיזי: מה-Driver אל כרטיס הרשת (NIC) מעל ה-PCIe**
כאן אנחנו עוברים מה-CPU אל ה-Bus של המחשב (PCI Express).
 - **The Doorbell (Register Access):** כרטיס הרשת הוא רכיב עצמאי. הדרייבר כותב ערך קטן (לרוב מספר ה-Descriptor האחרון שמולא) לתוך רגיסטר פיזי שנמצא על כרטיס הרשת. פעולה זו מכונה "Ring the Doorbell".
 - **DMA Engine (Direct Memory Access):** לכרטיס הרשת יש בקר DMA פנימי. ברגע שהוא קיבל את ה"צלצול", הוא הופך ל-Master של ה-PCIe Bus. הוא פונה לזיכרון ה-RAM של מחשב ה-room247 ו"מושך" את נתוני החבילה ישירות מה-RAM אל ה-Buffer הפנימי שלו על הכרטיס.
 - **מדוע זה חשוב?** ה-CPU של הסטודנט ב-room247 פנוי לחלוטין לבצע פעולות אחרות בזמן שהכרטיס "שואב" את המידע מהזיכרון.

4. עיבוד החומרה: בתוך ה-NIC עצמן

ברגע שהנתונים הגיעו לזיכרון הפנימי (SRAM) של הכרטיס:

- **Checksum Offloading**: אם ה-IP ביקש זאת, המעבד הייעודי על הכרטיס (Network Processor) ירוץ על כל הבתים ויחשב את ה-Checksum של ה-TCP ושל ה-IP.
- **Lso (Large Send Offload)**: אם ה-IP העביר הודעה גדולה מאוד (נניח 64KB), כרטיס הרשת עצמו יכול לחתוך אותה לסגמנטים של 1460 בתים (Segmentation Offload), ובכך לחסוך ל-CPU של המחשב עבודה רבה.
- **MAC Framing**: הכרטיס מוסיף את ה-Ethernet Header (MAC Address) ואת ה-CRC (בדיקת שגיאות לשכבה 2).
- **Parallel to Serial Conversion**: הנתונים מומרים מביטים מקבילים בזיכרון לזרם טורי (Serial) של אותות חשמליים או אופטיים שנשלחים דרך ה-PHY (השכבה הפיזית) אל הכבל.

5. האישור (The Completion Path)

כאשר הביט האחרון יצא מהכרטיס לכבל:

1. **Interrupt**: הכרטיס שולח סיגנל חשמלי (Interrupt) למעבד.
2. **ISR (Interrupt Service Routine)**: ה-Kernel עוצר לרגע, קורא את הדרייבר, והדרייבר מסמן בטבעת (TX Ring) שהתא פנוי.
3. **Return to IP**: ה-NDIS מודיע ל-IP שהחבילה "נשלחה" וה-IP יכול לשחרר את הזיכרון המקורי ששימש את ה-Chrome.